Operating Systems Lab Mini Project Report

Restaurant Management System using OS Concepts

Aryan Sharma BT2024212

April 30, 2026

1 Project Link

The complete source code is available at:
https://github.com/aryansharma0305/OS-Lab-Project-Sem4

2 Problem Statement

The objective of this project is to design and implement a multi-user Restaurant Management System that demonstrates core Operating Systems concepts such as concurrency, synchronization, inter-process communication, and socket programming.

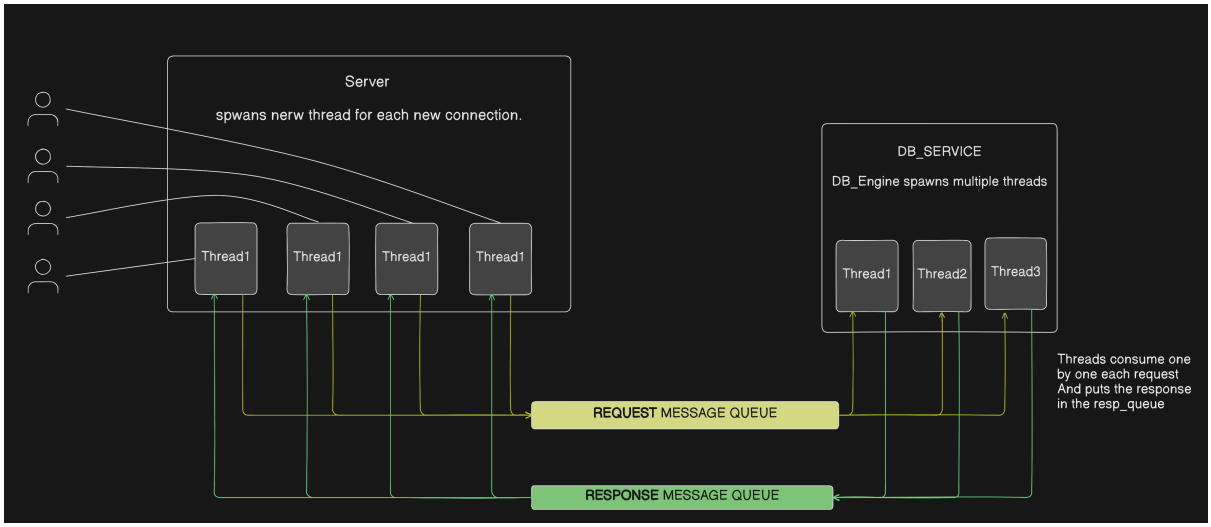
The system allows multiple clients (customers, chefs, admin) to interact with a central server to place orders, manage menus, and update order statuses while ensuring safe and consistent data handling.

3 System Overview

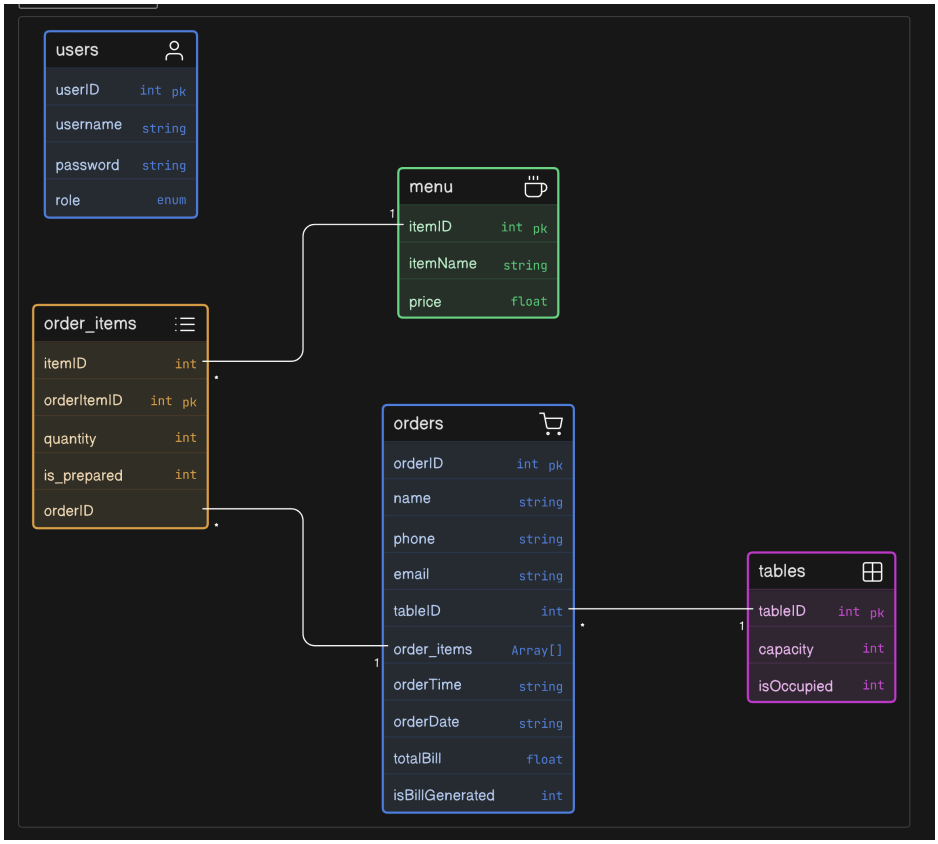
The system follows a client-server architecture:

- Clients interact with the server using TCP sockets
- The server handles multiple clients using threads
- A separate database service handles persistent storage
- Communication with the database service is done using IPC (message queues)

4 System Design



5 ER Diagram



6 Role-Based Authorization

The system defines multiple roles:

- **Admin**: Can add menu items, manage tables
- **Chef**: Can view and update order status
- **Customer**: Can place and view orders

Access control is enforced at the server level, ensuring each role can only perform permitted actions.

7 Socket Programming

The system uses TCP socket programming for communication:

- Server listens for incoming client connections
- Each client connects using a socket
- Server creates a new thread for each client

This allows multiple clients to interact with the system concurrently.

8 Concurrency Control

Concurrency is handled using:

- Threads for handling multiple clients
- Mutex locks to protect shared resources

This ensures that multiple users can perform operations simultaneously without conflicts.

9 Inter-Process Communication (IPC)

The system uses **System V Message Queues** for communication between:

- Server process
- Database service process

This separation ensures modularity and efficient communication between components.

10 Data Consistency

To maintain consistency:

- Critical sections are protected using mutex locks
- Shared data structures are accessed in a controlled manner

This prevents:

- Race conditions
- Lost updates

11 Implementation Details

- Language: C
- Communication: TCP sockets
- IPC: Message Queues
- Concurrency: POSIX threads
- Synchronization: Mutex locks

12 Challenges Faced

- Managing synchronization between multiple threads
- Designing efficient IPC between server and database
- Avoiding race conditions during concurrent access

13 Conclusion

This project successfully demonstrates practical implementation of Operating Systems concepts in a real-world application. It highlights how concurrency, synchronization, and communication mechanisms work together to build a robust multi-user system.

14 Demo Photos

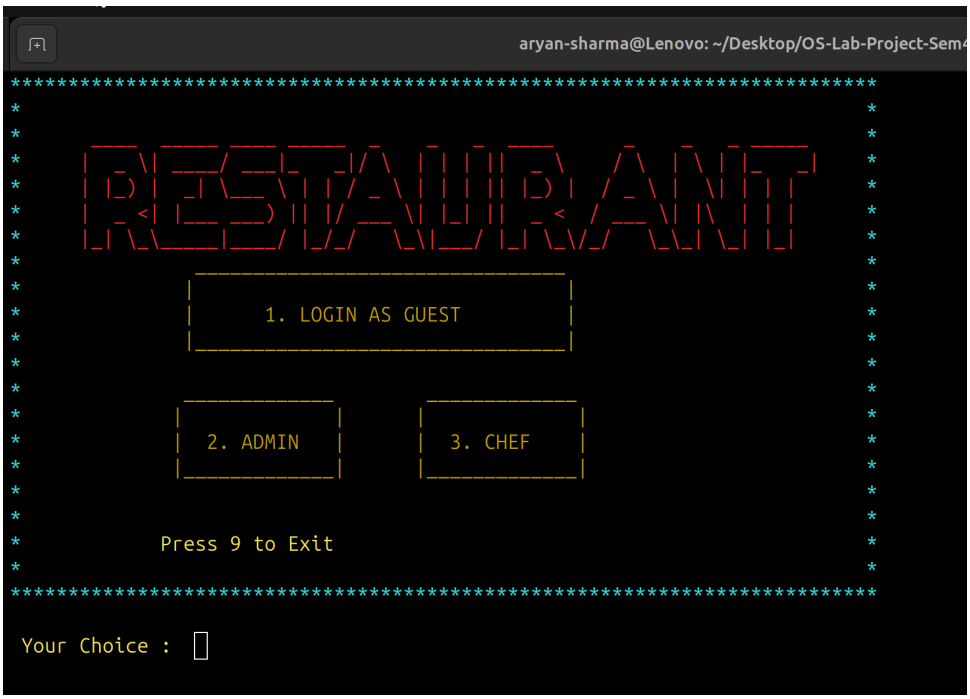


Figure 1: Demo Screenshot

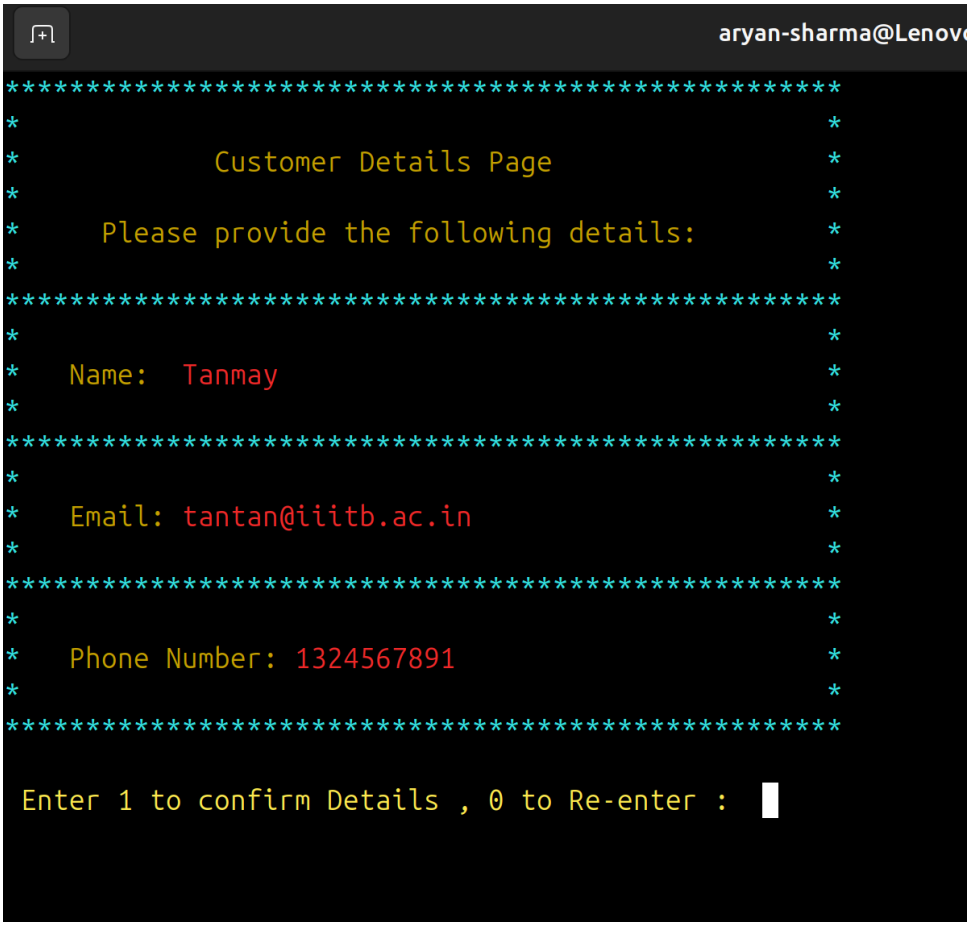


Figure 2: Demo Screenshot

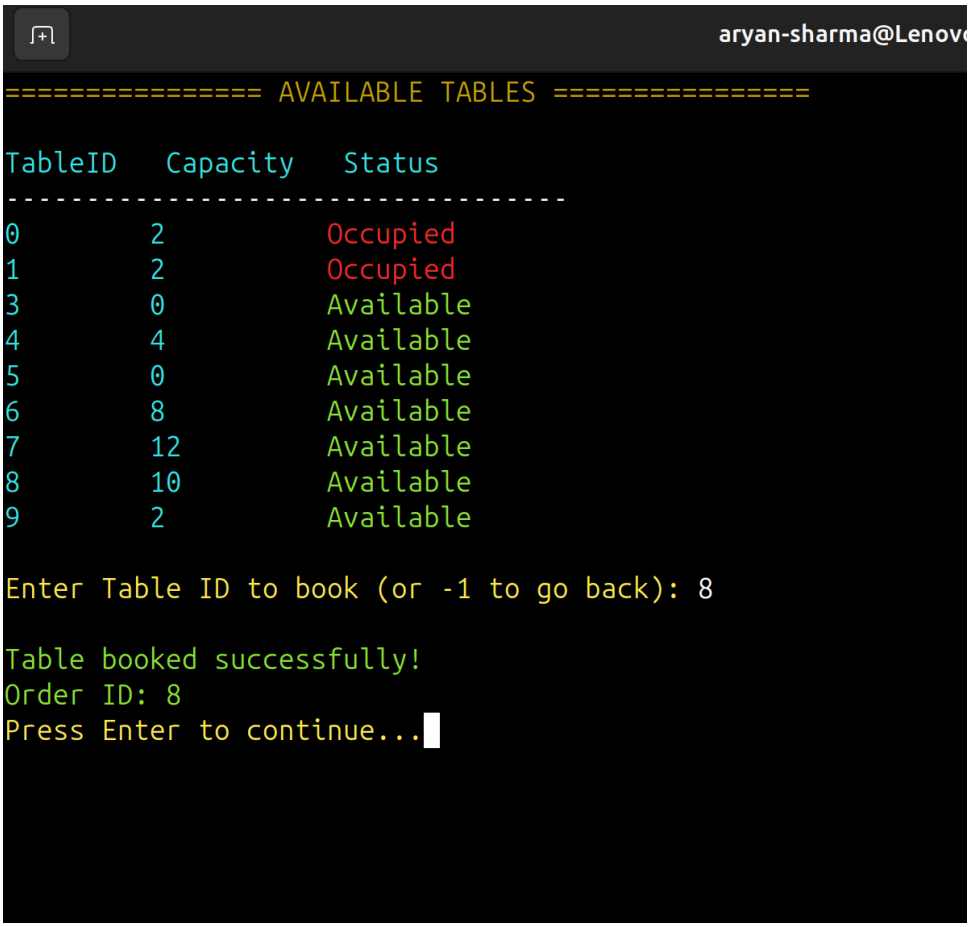


Figure 3: Demo Screenshot

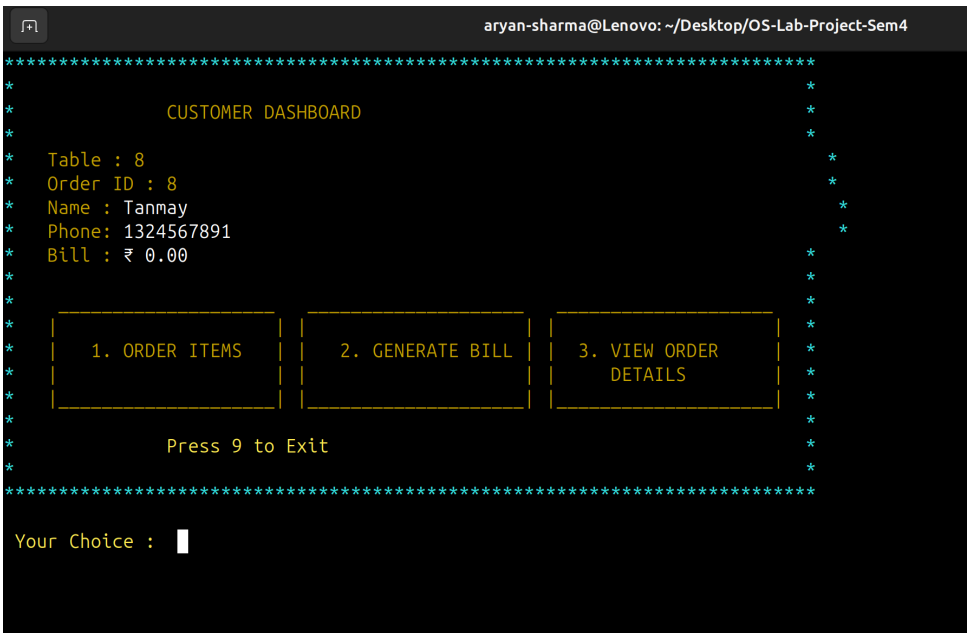


Figure 4: Demo Screenshot

Figure 5: Demo Screenshot

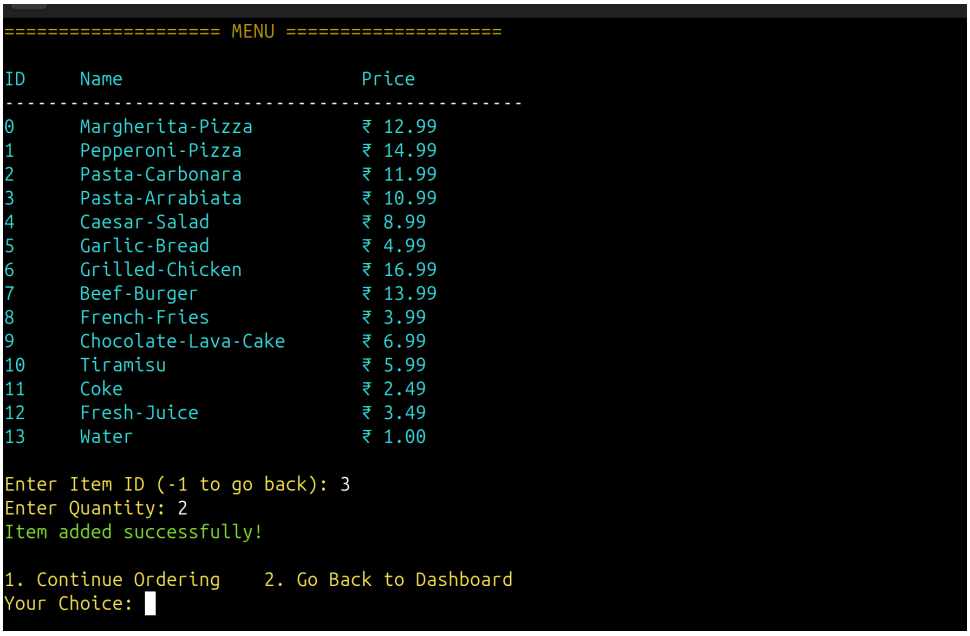


Figure 6: Demo Screenshot

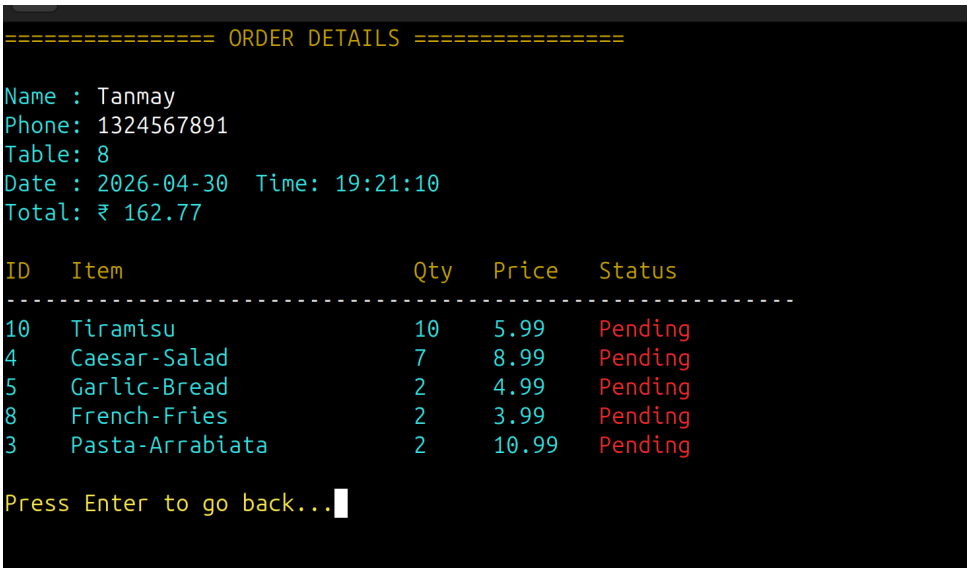


Figure 7: Demo Screenshot

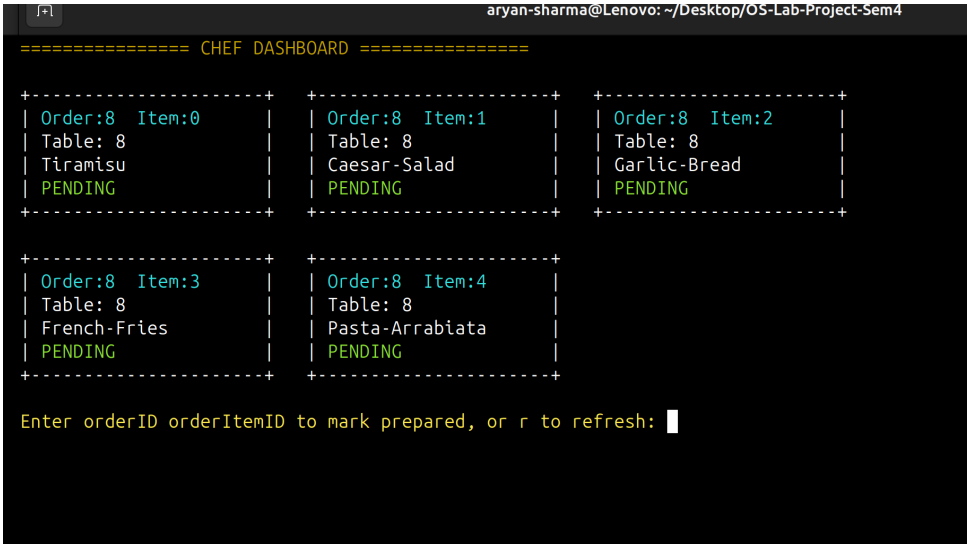


Figure 8: Demo Screenshot

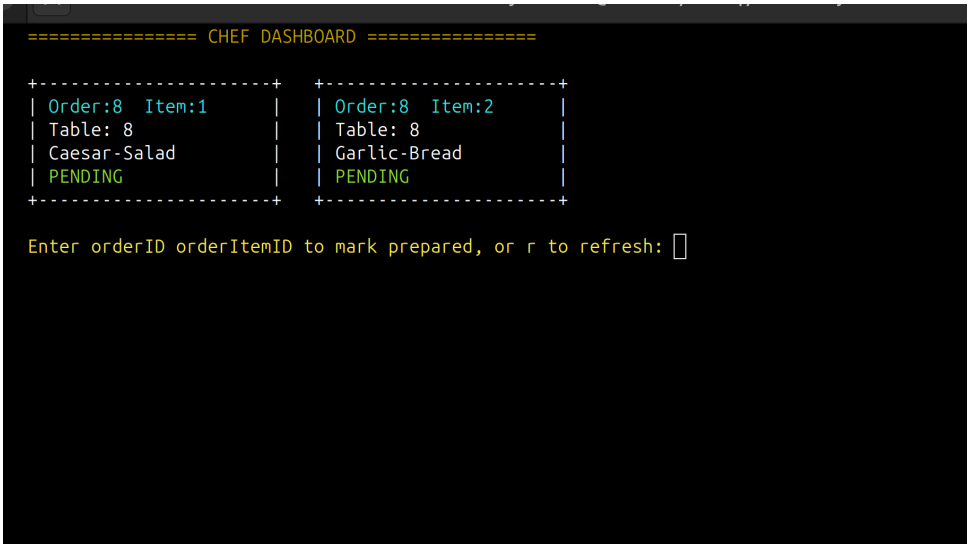


Figure 9: Demo Screenshot

Figure 10: Demo Screenshot

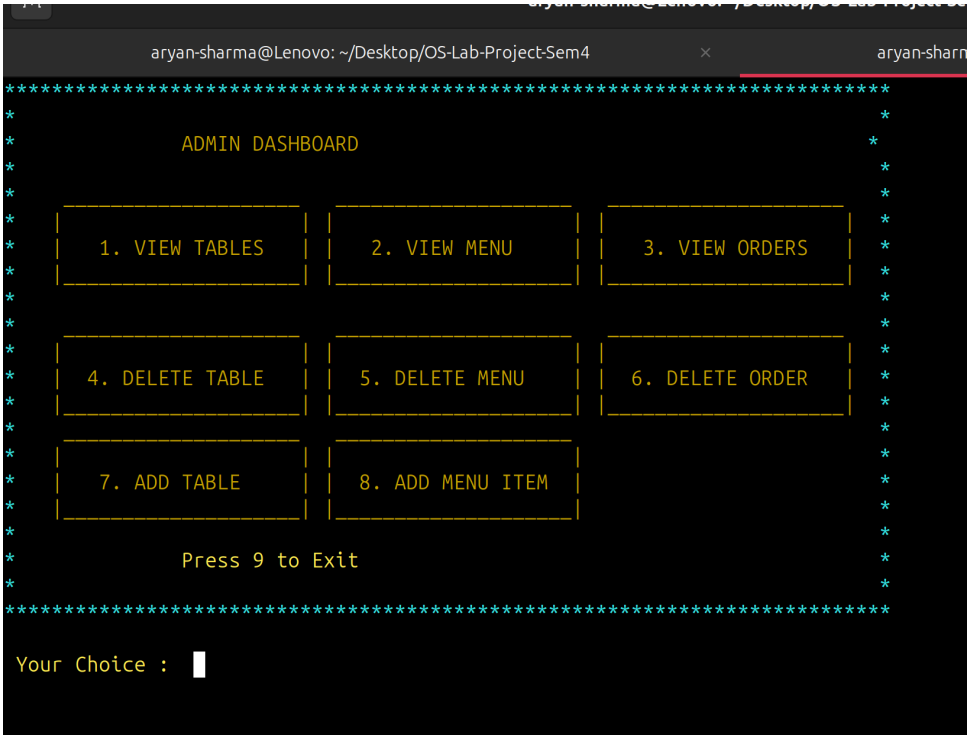


Figure 11: Demo Screenshot